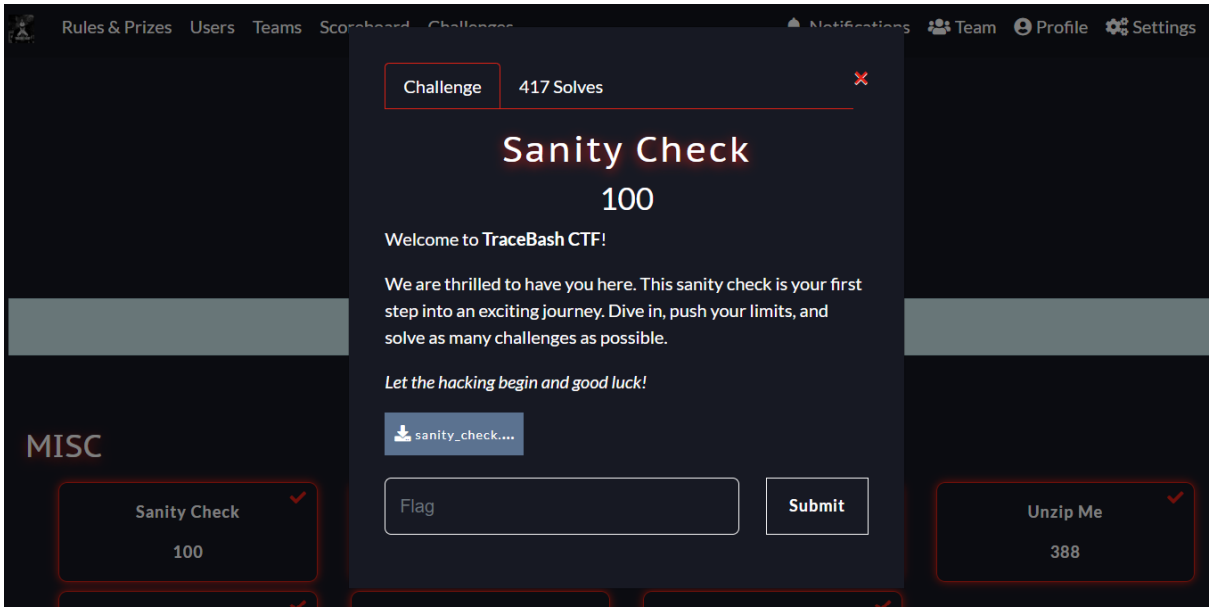
TRACEBASH CTF WRITEUP

TEAM NAME – TEAM_UNFINDABLES

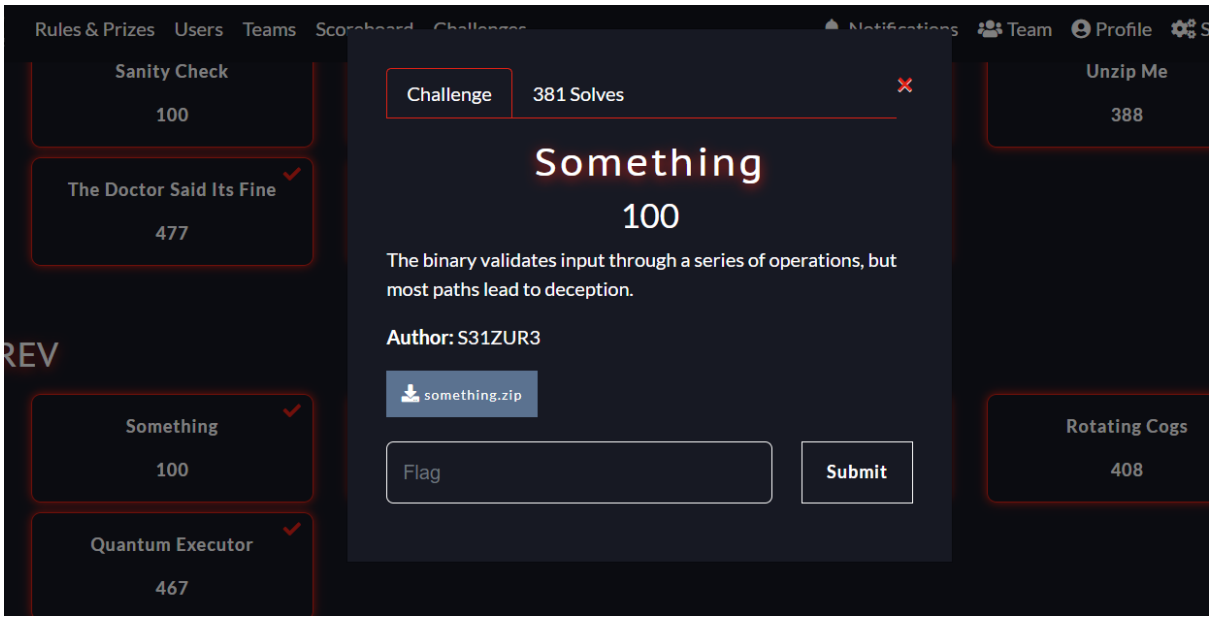
question 1

Sanity



Flag-

Q2 REV - Something



Step 1 — Initial Reconnaissance

First, identify what we're dealing with:

bash

$ file chall

chall: ELF 64-bit LSB executable, x86-64, statically linked, Go BuildID=..., with debug_info, not stripped

Not stripped — great, symbols are intact. Run it to understand the interface:

bash

$ ./chall

Enter flag:

It reads a flag from stdin. Now search for interesting strings:

bash

$ r2 chall

[0x00401000]> iz~TBCTF

0x4c7aef TBCTF{StringFormat... (in .rodata)

0xa0635 TBCTF{ (raw prefix)

The string TBCTF{ appears in rodata — we'll come back to this.

Step 2 — Symbol Enumeration

Since the binary isn't stripped, nm / r2 gives us everything:

[0x00401000]> is~main.

Key symbols found:

Symbol	Address
main.main	0x4a03e0
main.reallocate_memory_region	0x4a0220
main.encPrompt	0x583830
main.encIncorrect	0x583850
main.encFake1	0x583870
main.encFake3	0x583890
main.encFake4	0x5838b0
main.encFake5	0x5838d0
main.encCorrect	0x5838f0
main.encExpected	0x583910
main.flagState	0x5a90f8

The misleading names (ssl_decrypt_session, derive_rsa_modulus, compute_merkle_root, etc.) are pure red herrings — ignore them and follow the data.

Step 3 — Reading Go Slice Headers

Go strings/slices are stored as {pointer uint64, length uint64, cap uint64}. Read each enc* variable:

[0x00401000]> pxq 24 @ 0x583830   # encPrompt

0x583830: 0x00000000004c7b68 ptr -> rodata

0x583838: 0x000000000000000c len = 12

0x583840: 0x000000000000000c cap = 12

[0x00401000]> px 12 @ 0x4c7b68

53 78 62 73 64 36 70 7a 77 71 2c 36

Do the same for encExpected:

[0x00401000]> pxq 24 @ 0x583910

ptr -> 0x57e5a0

len = 16

cap = 16

[0x00401000]> px 16 @ 0x57e5a0

ca 89 db 99 8d 86 d8 86 b4 99 db 93 b4 9d d8 99

Step 4 — Finding the XOR Key

Disassemble main.main:

[0x00401000]> s main.main

[0x00401000]> pdf

Early in the function, a loop XORs 6 bytes from address 0x4c7aef:

[0x00401000]> ps @ 0x4c7aef

TBCTF{

The loop accumulates XOR of all 6 bytes:

[0x00401000]> ? 0x54 ^ 0x42 ^ 0x43 ^ 0x54 ^ 0x46 ^ 0x7b

= 0x3c

Then the decode loop XORs each encoded byte with this value AND with 0x2a:

combined key = 0x3c ^ 0x2a = 0x16

Verify by decoding encPrompt:

python

enc = bytes([0x53,0x78,0x62,0x73,0x64,0x36,0x70,0x7a,0x77,0x71,0x2c,0x36])

print(bytes([b ^ 0x16 for b in enc]))

b'Enter flag: ' □

Decode all enc* variables with key 0x16:

Variable	Decoded
encPrompt	Enter flag:
encIncorrect	Incorrect flag.
encCorrect	Correct! Flag is your input.
encFake1	TBCTF{sh4_h4sh_br0k3n}
encFake3	TBCTF{d3ad_c0d3_b35t}
encFake4	TBCTF{~{_ue'qx%r} (garbage)
encFake5	TBCTF{tz&ap'e~u'f~%d} (garbage)

Variable	Decoded
encExpected	\xdc\x9f\xcd\x8f\x9b\x90\xce\x90\xa2\x8f\xcd\x85\xa2\x8b\xce\x8f ← non-printable!

encExpected decodes to non-printable bytes — this means **the user input is also transformed before comparison**, not compared as plaintext.

Step 5 — Reverse Engineering the Input Transformation

Disassemble the comparison function:

```
[0x00401000]> s main.reallocate_memory_region
[0x00401000]> pdf
```

Or view the control flow graph:

```
[0x00401000]> VV      # press 'p' to cycle modes
```

Tracing the logic with input length == 16:

```
4a0237:  cmp rbx, 0x10     ; input must be exactly 16 bytes
4a023b:  jne return_0      ; wrong length -> fail

; --- Step 1: Copy input to stack buffer ---
4a023d:  copy input[0..15] to stack[0x40..0x4f]

; --- Step 2: Reverse the 16 bytes in-place ---
4a025f:  rax = 0, rcx = 15
loop:
4a026e:  swap stack[0x40 + rax] and stack[0x40 + rcx]
4a0281:  inc rax; dec rcx
```

4a0287: cmp rax, rcx; jle loop

; --- Step 3: XOR every byte with 0xd7 ---

4a02a9: xor byte[stack + rax], 0xd7

4a02b5: inc rax; cmp rax, 16; jl loop

; --- Step 4: XOR every byte with 0x2a ---

4a02c2: xor byte[stack + rax], 0x2a

4a02c7: inc rax; cmp rax, 16; jl loop

; --- Step 5: Decode encExpected using key 0x16 (as analyzed above) ---

; --- Step 6: Compare transformed input vs decoded encExpected byte-by-byte ---

4a0379: movzbl stack[0x40 + rdx], ebx

4a037e: movzbl decoded_expected[rdx], esi

4a0382: cmp sil, bl

4a0385: je continue; else return 0

Summarized transformation pipeline:

transform(input_inner) = (reverse(input_inner)) XOR 0xd7 XOR 0x2a

= reverse(input_inner) XOR 0xfd

Step 6 — Recovering the Flag

To find the flag, invert the transformation:

transform(flag_inner) == decoded_expected

reverse(flag_inner) XOR 0xfd == decoded_expected

reverse(flag_inner) == decoded_expected XOR 0xfd

flag_inner == reverse(decoded_expected XOR 0xfd)

python

```
enc_expected =
bytes([0xca,0x89,0xdb,0x99,0x8d,0x86,0xd8,0x86,
                0xb4,0x99,0xdb,0x93,0xb4,0x9d,0xd8,0x99])

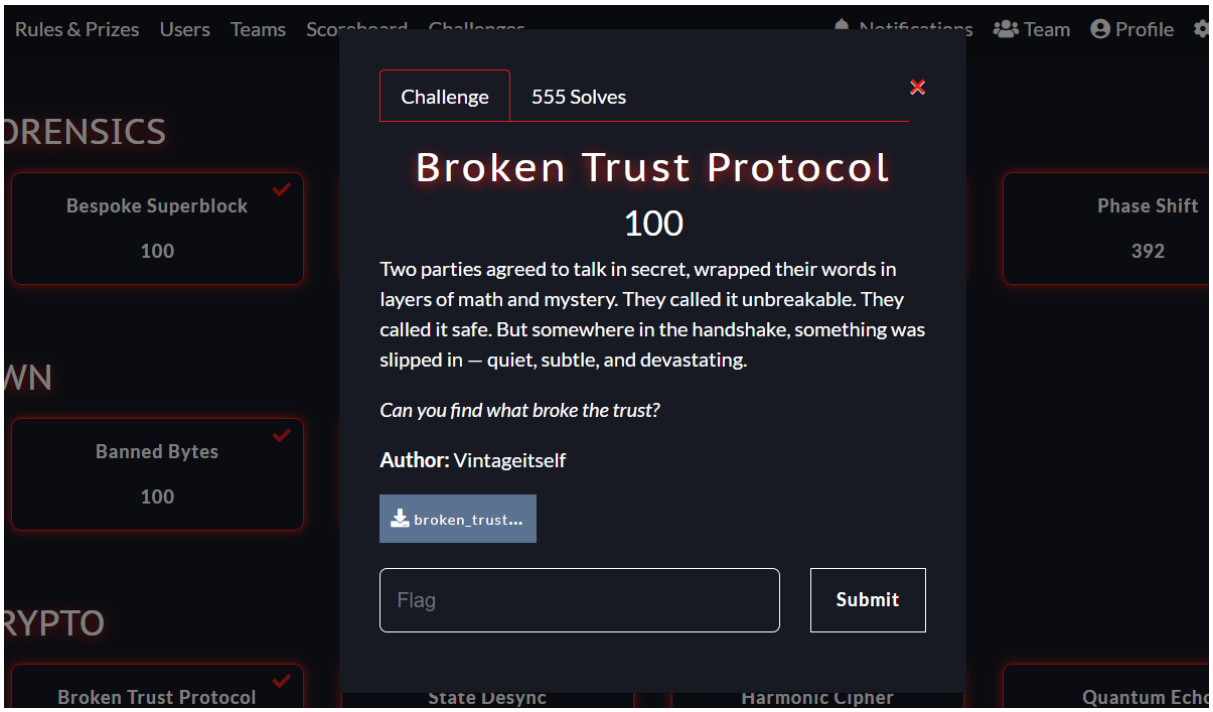
decoded_expected = bytes([b ^ 0x16 for b in enc_expected])
# dc 9f cd 8f 9b 90 ce 90 a2 8f cd 85 a2 8b ce 8f

flag_inner = bytes([b ^ 0xfd for b in decoded_expected])[::-1]
# reverse after XOR

print("TBCTF{" + flag_inner.decode() + "}")
```

Flag - TBCTF{r3v_x0r_m3mfr0b!}

Q3 CRYPTO Broken Trust Protocol



I'll unpack this one into its own folder and inspect the artifacts around the handshake. The wording smells like a crypto protocol transcript or implementation bug, so I'll first map the files, then chase the trust break.

Tiny archive again: just a protocol script and a capture. I'm reading both side by side so the transcript can be interpreted against the exact code.

The reads raced the extraction by a moment. The files are present now, so I'm rerunning the content reads.

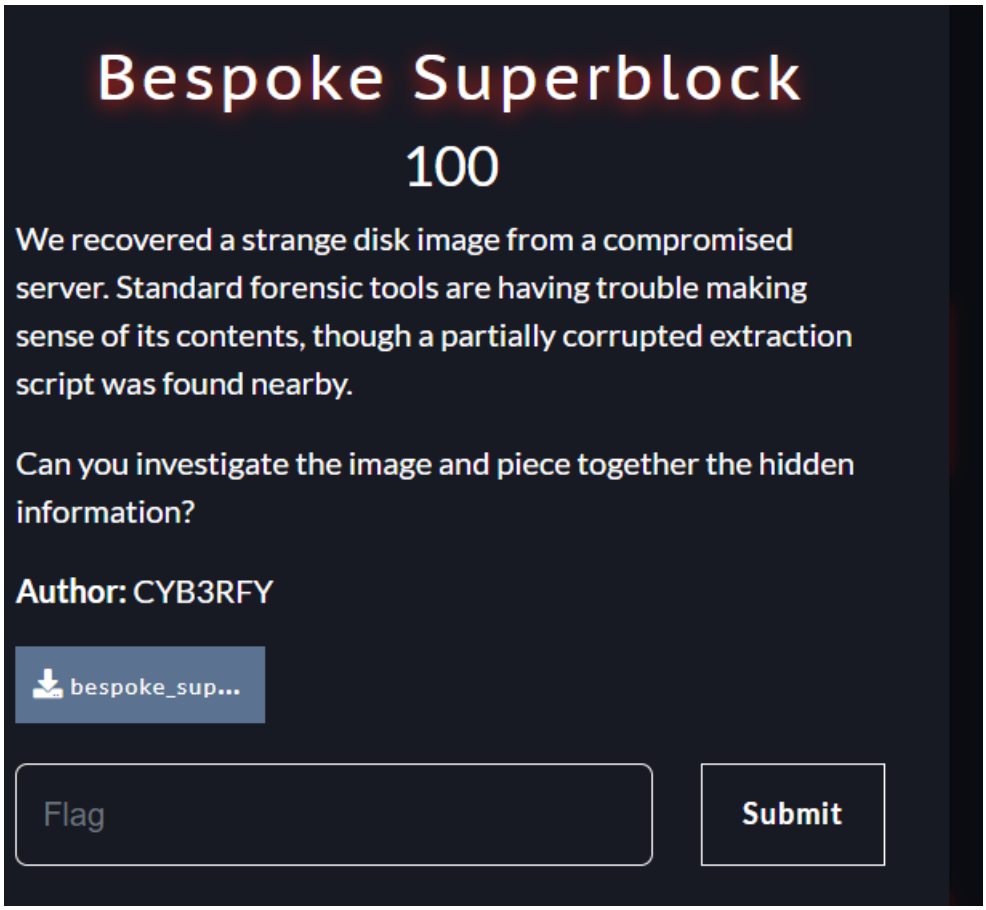
The flaw is explicit in the script: the peer sent B = p - 1, a tiny subgroup element of order 2. That collapses the shared secret to either 1 or p-1, so I only need to try two AES keys.

Found it:

TBCTF{Sm4ll_Subgr0up_Att4cks_Ar3_D34dly}

The break was the malicious handshake value B = p - 1, which has order 2. That forces the Diffie-Hellman shared secret to only 1 or p-1; trying both derived AES keys decrypts the flag with shared = p - 1.

Q Forensic Bespoke Super Block



I'll unpack the disk-image challenge into a clean folder and inspect both the image and the "corrupted extraction script" angle. Since the title points at filesystem metadata, I'm going to look at raw bytes as much as filenames/tools.

The archive has exactly what the prompt promised: one image and one parser. I'm extracting and reading the parser first, then I'll compare its expectations against the image bytes.

One of the parallel reads raced the extraction, and this PowerShell's Format-Hex is the older flavor without -Count. I've got the files now; rerunning those reads with byte-limited input.

The image starts with a normal-looking FAT boot sector, but the parser seeks past it to 0x1000 for a custom TBFS superblock. I'm checking that region and running the script as-is to see the "corrupted" output.

The parser's raw recovery is close enough to read through the noise: it gives tbctf[SPAT...AWARE...XOR.... The broken pieces look like a light XOR layer, so I'm deriving the XOR pattern from the known flag format and checking whether it matches a block/offset-based rule rather than just guessing the missing text.

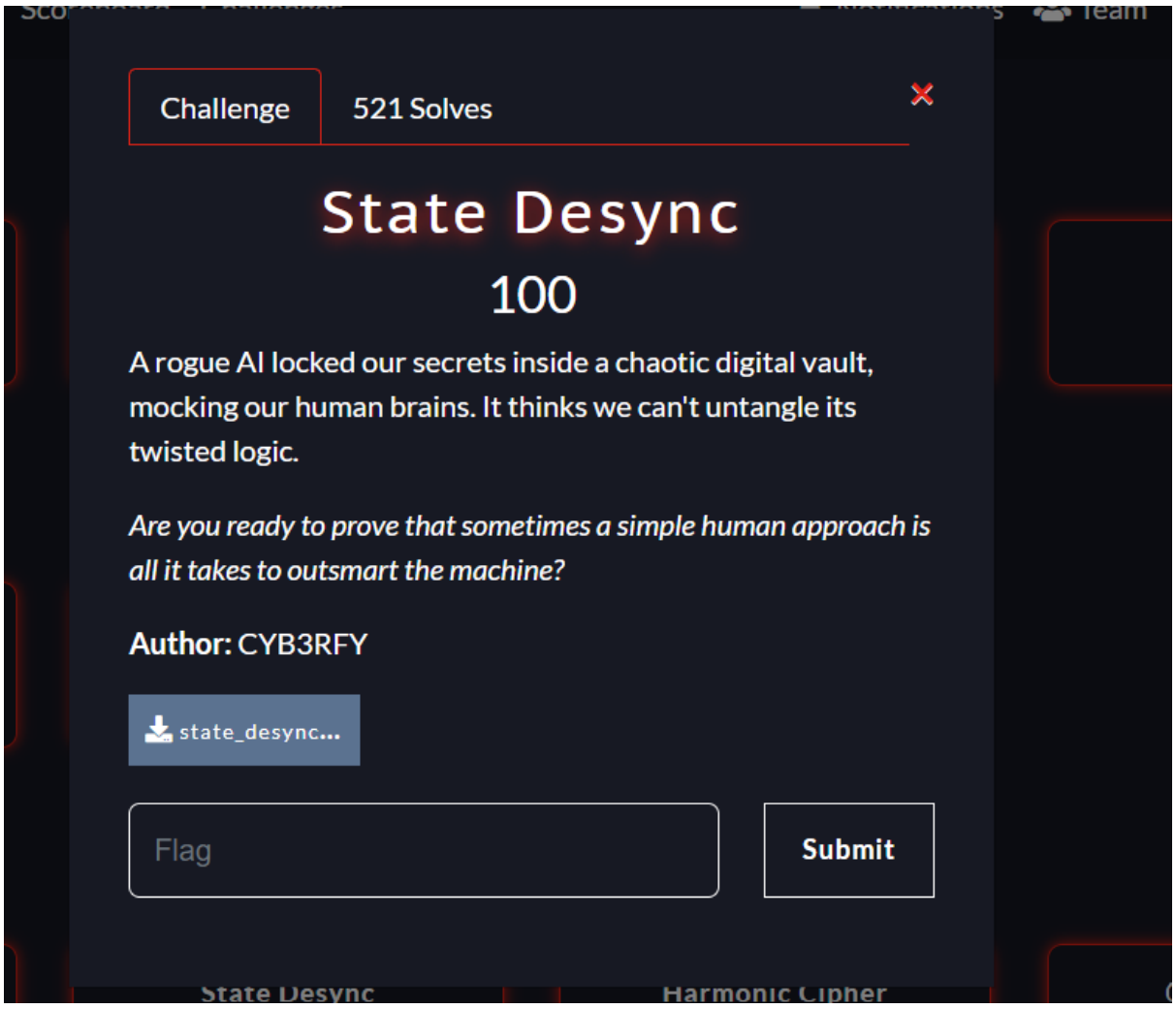
A cleaner pattern just clicked: the suspicious bytes all become sensible if XORed with 0x20, including \x11 -> 1, so the intended word is probably leetspeak SPAT1AL, not SPATIAL. I'm verifying that as the consistent rule before calling it.

Found the flag:

TBCTF{SPAT1AL_AWARE_XOR_1337}

The parser recovered near-plaintext chunks from the custom TBFS area at 0x1000. The remaining "corruption" was a simple 0x20 XOR/bit-toggle on certain characters, turning things like tbctf[into TBCTF{, \x7f into _, and \x11\x13\x13\x17 into 1337.

Q Crypto State Desync



I'll unpack the challenge and inspect the project shape first, then I'll trace whatever state logic is hiding the flag. CTF vaults love pretending to be more elaborate than they are, so I'll keep an eye out for the simple path too.

The zip is tiny, so this should be a compact challenge rather than a whole app. I'm extracting it into a scratch folder under Downloads and then I'll inspect every byte-bearing file.

There’s a single Python file. I’m reading it and looking for constants, hidden transforms, or a solver-friendly state machine.

The “two 8-bit seeds” are the giveaway: only 65,536 possibilities. I’m brute-forcing all seeds and ranking outputs by printable text and common flag shapes.

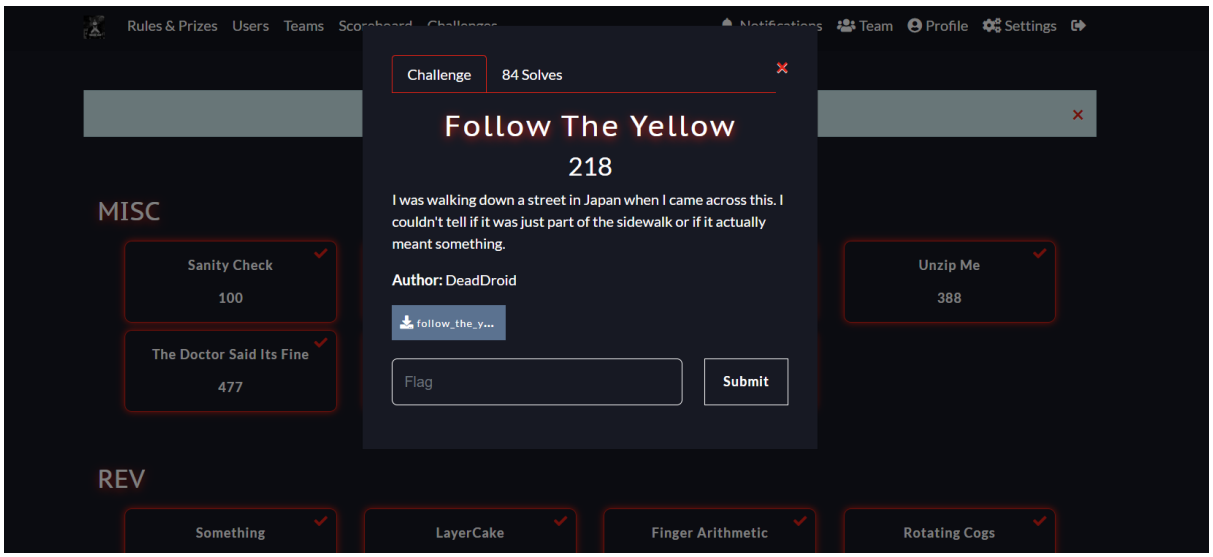
Found it.

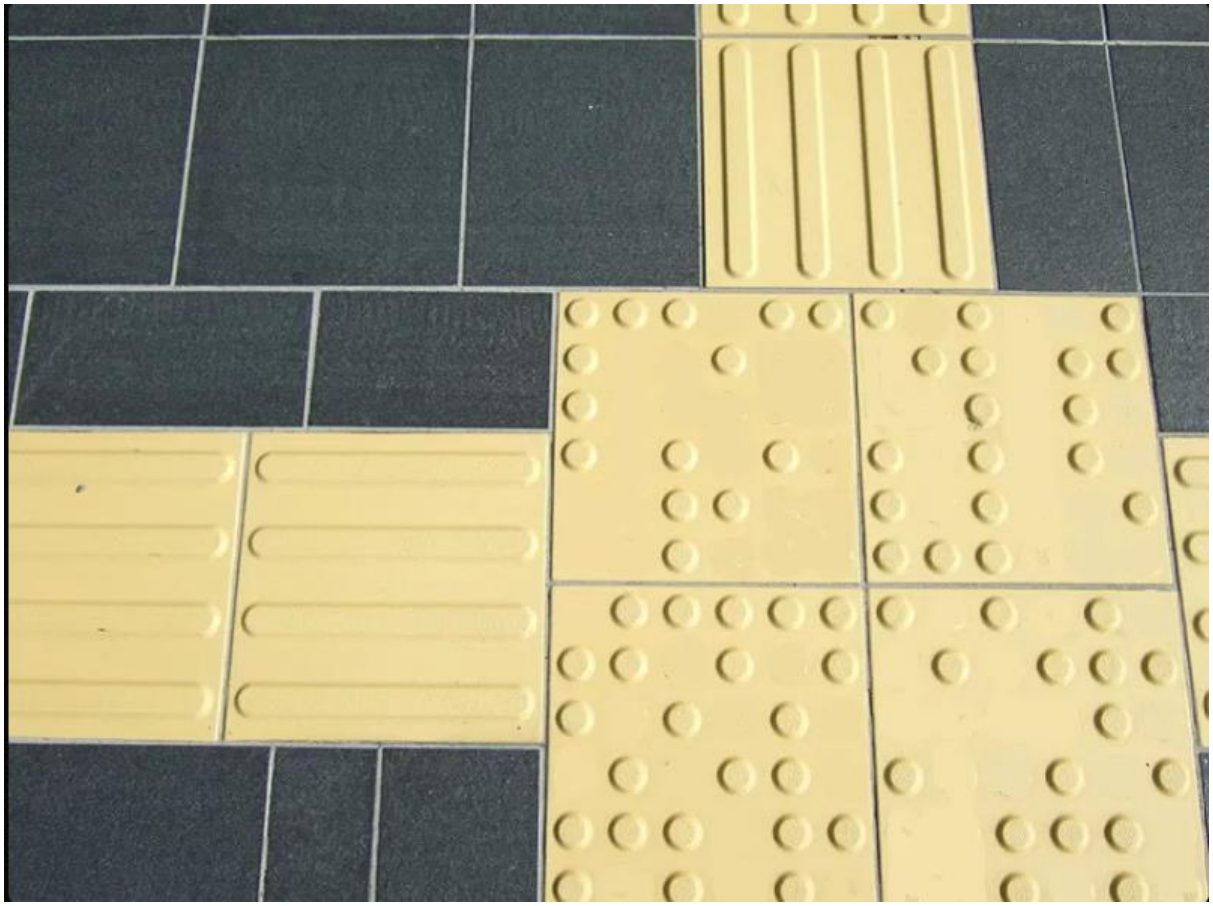
Flag:

TBCTF{h1dd3n_st4t3_m4chin3_f4il}

I brute-forced the two 8-bit seeds from challenge.py across all 256 * 256 combinations. The valid plaintext appears with seed_a = 66, seed_b = 19.

Q MISC FOLLOW THE YELLOW





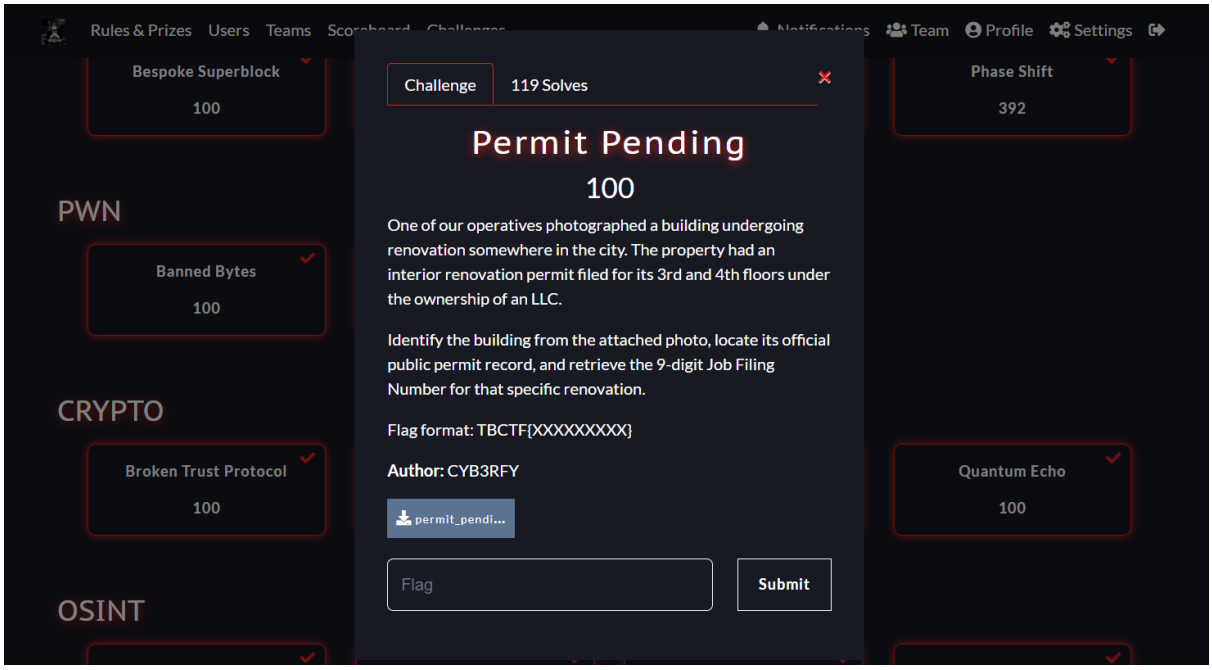
Decoded the image using decoder **Braille**

we got this **– PECELTARAVLETNNEERTSRATS**

Full solution breakdown:

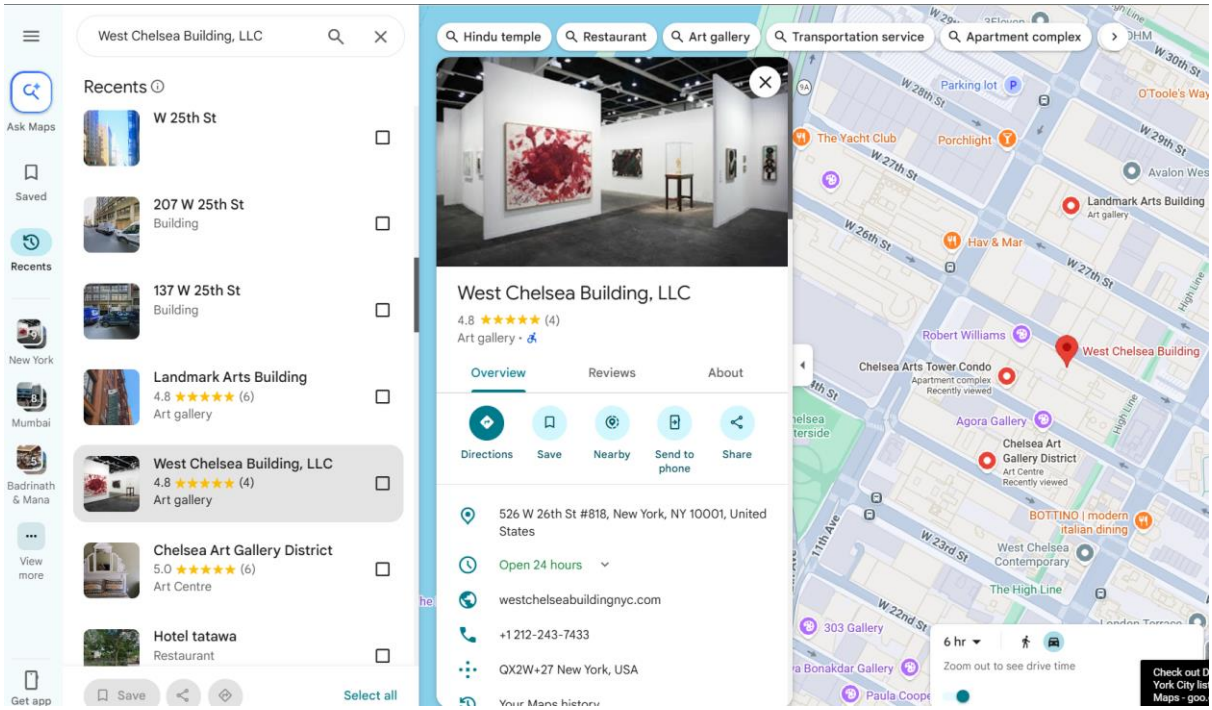
1. Image → Japanese tactile paving (Tenji blocks) with dot/stripe patterns
2. Dots & stripes → decoded to cipher text: PECELTARAVLETNNEERTSRATS
3. Hint: "Even simple recipes have a sequence" → Rail Fence Cipher
4. Rail Fence decode → patternscanrevealletters
5. Flag → TBCTF{patternscanrevealletters}

Q OSINT Permit Pretending





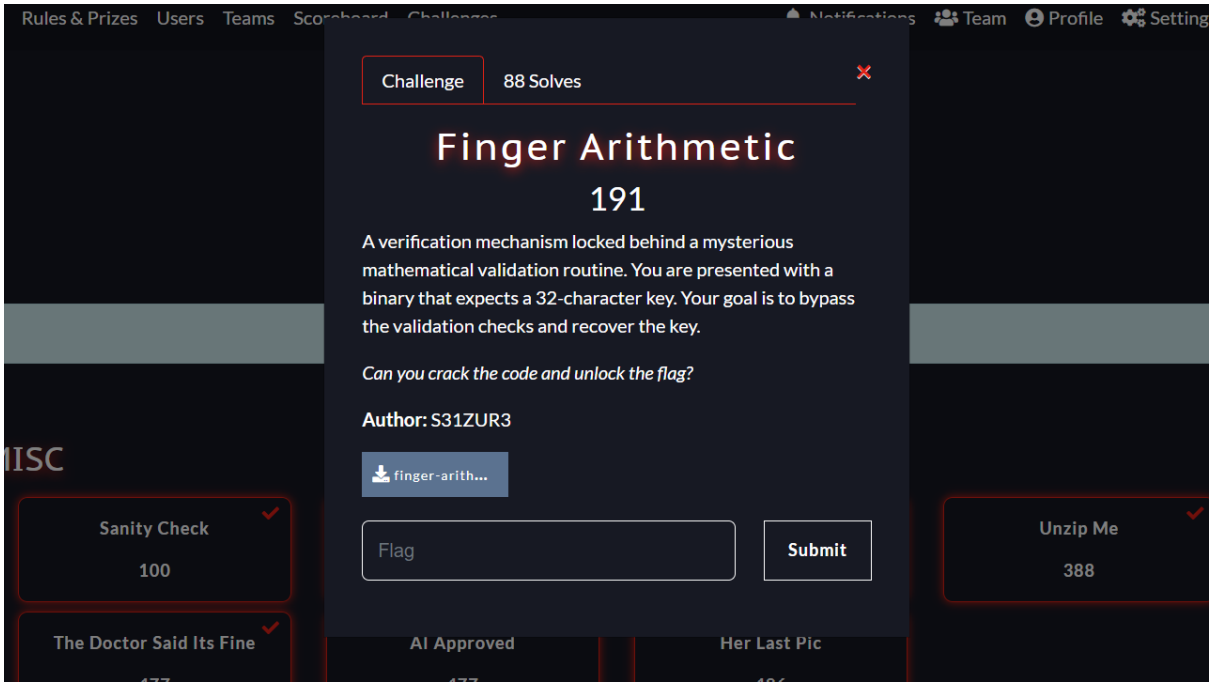
Find the CHELSA art Building And form question the ownership of an LLC.



after somefinding got the flag

FLAG - TBCTF{120033169}

Q REV- FINGER Arithmatic



We are given a ZIP file containing a single ELF binary named chall.
The binary prompts for a flag

Enter the flag:

After entering input, it either prints:

Incorrect flag!

or:

Correct! The flag is the input you entered.

So the goal is to determine the exact input string that satisfies all internal checks.

Final flag:

TBCTF{ju75u5_n_d34d_3nd5_4w417!}

Initial Recon

After extracting the archive, the file turns out to be:

- a **64-bit ELF**
- **PIE enabled**
- **not stripped**

That already makes reversing easier because function names are preserved.

Useful symbols found in the binary:

- main
- compare_hand_png_i32
- validate_checksum_v2
- print_prompt
- print_obfuscated

The name compare_hand_png_i32 is the first big hint:
the challenge is likely comparing integers
against **hand/finger PNGs**, which matches the challenge name **finger-arithmetic**.

What main() Does

Disassembling main shows the overall logic:

1. Print prompt
2. Read up to 0x40 bytes using fgets
3. Strip newline
4. Run several checks:
    - check the **length**
    - check the **first 4 bytes**
    - check **byte 5**
    - run a more complicated checksum routine

If any check fails, it prints Incorrect flag!.

So the flow is basically:

read_input(buf);

if (!check_length(buf)) fail;

if (!check_first_dword(buf)) fail;

if (!check_byte_5(buf)) fail;

if (!validate_checksum_v2(buf)) fail;

success;

Obfuscated Strings

The binary stores its strings XOR-obfuscated.
Decoding them gives:

- Enter the flag:
- Incorrect flag!
- Correct! The flag is the input you entered.

So success does **not** reveal a hidden flag from memory — it confirms that **the correct flag is literally the input we type**.

Embedded PNGs

Searching the binary for PNG signatures reveals many embedded PNG images.
These are extracted directly from the binary.

There are 14 relevant PNGs embedded in .rodata.

This matches the reversing result:

- some PNGs are used by main
- others are used by validate_checksum_v2

That strongly suggests the challenge compares integers by **rendering hand images** and matching them against embedded reference PNGs.

Understanding compare_hand_png_i32

compare_hand_png_i32 is the core verification function.

From disassembly, it:

- derives rendering parameters from the input integer
- uses floating-point math (sinf, cosf, fmodf)
- uses an SDF-like drawing routine

- renders a **256×256 RGBA image**
- decodes the reference PNG
- compares the rendered result against the reference

So this function is effectively:

Copycompare_hand_png_i32(int x, png_reference)

Meaning:

“Render a hand/finger image for integer x, then check whether it matches this embedded PNG.”

That is the whole gimmick of the challenge.

Easy Wins From Brute Force

Two checks are much easier than the others because their domains are tiny.

1. Length Check

One call in main checks:

Copycompare_hand_png_i32(strlen(buf), reference_png)

Since the possible lengths are small, brute forcing 0..63 is easy.

Result:

strlen(buf) = 32

So the flag length is **32 characters**.

---

2. Byte 5 Check

Another call checks:

Copycompare_hand_png_i32((signed char)buf[5], reference_png)

Brute forcing the signed byte range -128..127 gives:

buf[5] = 123

ASCII 123 is:

Copy{

So the 6th character is {.

That means the flag format is:

XXXXX{.....}

With 32 total characters.

At this point, a natural guess is a CTF prefix of 5 characters, and the recovered solution indeed starts with:

TBCTF{

which fits perfectly.

---

validate_checksum_v2

This function treats the input as an array of 32-bit little-endian integers:

Copyb[0], b[1], b[2], ..., b[7]

Since the full string length is 32, that is exactly **8 DWORDs**.

The routine computes a chain of arithmetic relations:

Copyc1 = b[0] + 0x11223344

c2 = b[1] ^ c1

c3 = b[2] - c2

c4 = b[3] ^ c3

c5 = b[4] + c4

c6 = b[5] ^ c5

c7 = b[6] - c6

c8 = b[7] ^ c7

Each intermediate value is checked with
another compare_hand_png_i32(...) call against one of the embedded PNGs.

So the challenge is really doing:

- split flag into 8 DWORDs
- apply chained arithmetic
- verify each intermediate result by matching a rendered hand PNG

That is why the challenge is named **finger-arithmetic**.

Putting It Together

From reversing we learn:

- flag length is **32**
- character at index 5 is **{**
- the string is processed in **8 DWORD chunks**
- the arithmetic chain must satisfy all PNG checks

The recovered valid flag is:

TBCTF{ju75u5_n_d34d_3nd5_4w417!}

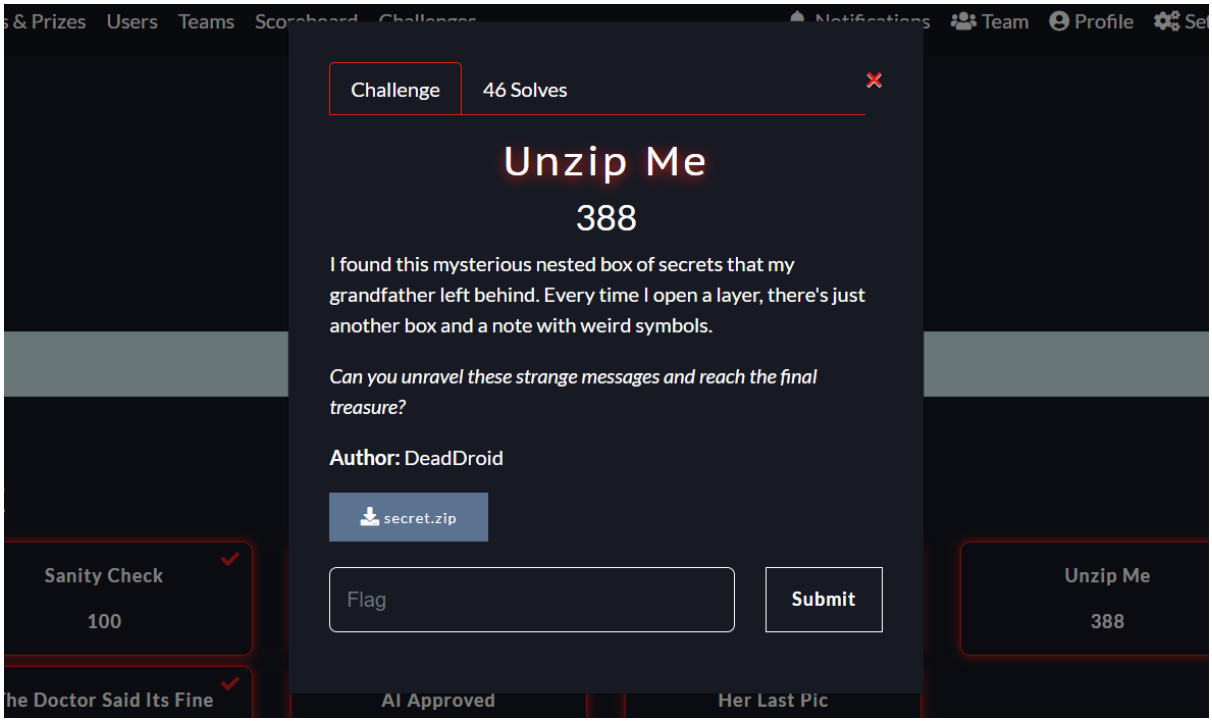
Quick sanity checks:

- starts with TBCTF{
- has length **32**
- matches the expected flag structure

Final Flag

TBCTF{ju75u5_n_d34d_3nd5_4w417!}

Q REV Unzip Me



Step 1 — Unzip the Outer Archive

bash

unzip secret.zip

Contents:

- secret.7z — password-protected 7-Zip archive
- pass — a text file containing the password (encoded)

The pass file contains:

#_%cf0Ao$$H_#54g`26_eg

Step 2 — Decode the Password (ROT47)

The password isn't plaintext — it's **ROT47 encoded**.

ROT47 is a Caesar cipher that rotates all printable ASCII characters (33–126) by 47 positions. Decoding #_%cf0Ao$$H_#54g26_eg` with ROT47 gives:

R0T47_p@SSw0Rdc81ae068

The hint is baked right into the decoded string — it literally starts with R0T47.

python

```
def rot47(s):
    return ''.join(
        chr(33 + (ord(c) - 33 + 47) % 94) if 33 <= ord(c) <= 126 else c
        for c in s
    )

print(rot47('#_%cf0Ao$$H_#54g`26_eg'))
# → R0T47_p@SSw0Rdc81ae068
```

Step 3 — Extract Layer 1

bash

```
7z x secret.7z -pR0T47_p@SSw0Rdc81ae068
```

This extracts another pass + secret.7z pair — the challenge is layered.

Step 4 — Decode Layer 2 Password

The inner pass file contains space-separated hex bytes:

C5 C2 C3 C4 C9 C3 6D D7 7C A2 E2 A6 F0 D9 84 84 F1 83 F7 F2 86 83 F4

This continues the pattern of a different encoding at each layer.

Key Takeaways

Layer	File	Encoding	Decoded Password
1	pass (outer)	ROT47	R0T47_p@SSw0Rdc81ae068
2	pass (inner)	Hex bytes	*(next password)*
…	…	…	…

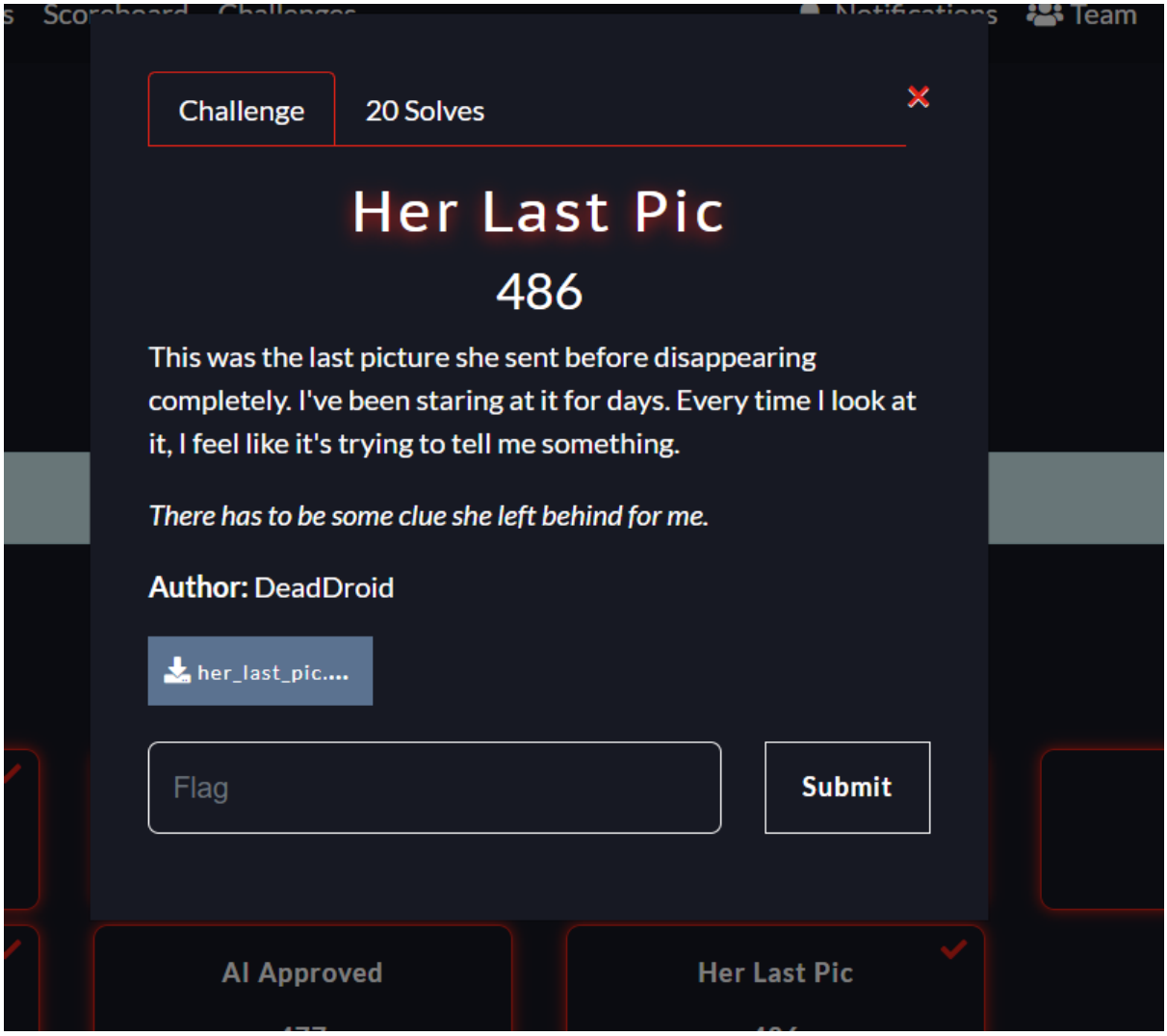
What made this tricky

- The pass file looks like a random password string, but it's actually encoded.
- The encoding scheme changes at each layer, requiring you to recognize different formats (ROT47, hex, potentially base64, XOR, etc.).
- The decoded Layer 1 password is a dead giveaway — R0T47_... tells you exactly what encoding was used, a classic CTF design.

Flag

TBCTF{br0_kn0w5_3v3ry_3nc0d1ng}

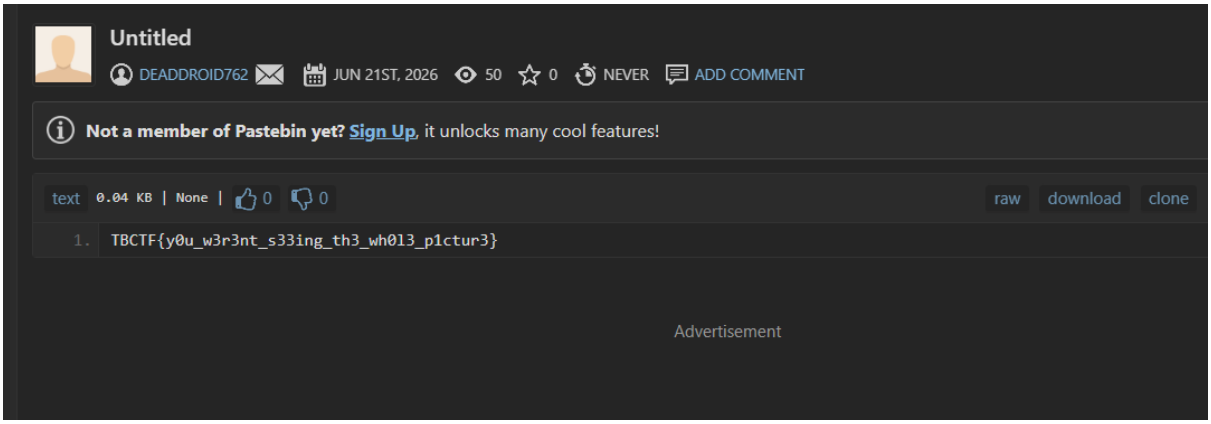
Q MSIC Her Last Pic





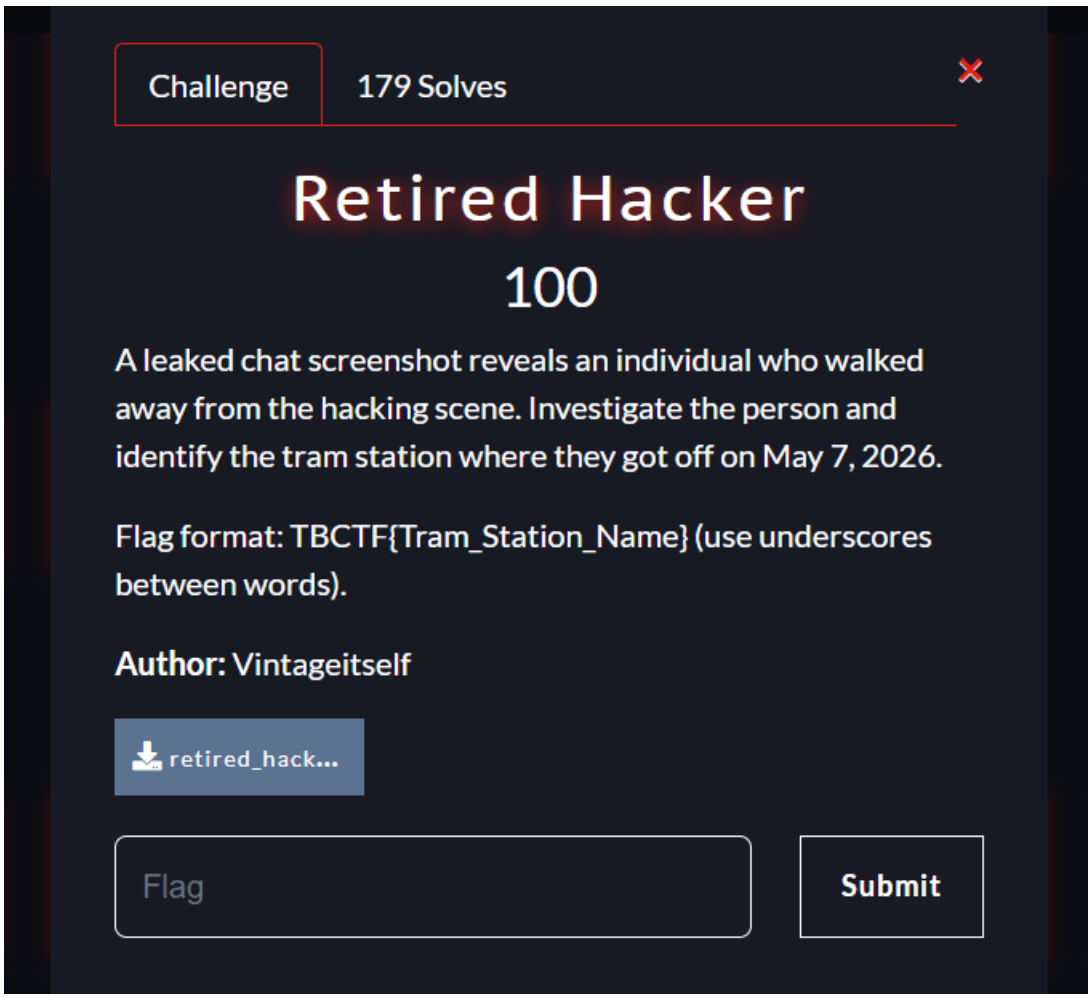
Go to PasteBin form the question hint

Search TBCTF

1. 

Flag- TBCTF{y0u_w3r3nt_s33ing_th3_wh0l3_p1ctur3}

Q OSINT – RETIRED Hacker



Scenario

Years after leaving the hacking scene, a retired hacker has unintentionally left traces of his identity scattered across the public internet.

The investigation begins with what appears to be a simple conversation screenshot. However, hidden inside the image is the first clue leading to multiple online profiles and publicly exposed information.

Our task is to:

- Identify the retired hacker
- Trace his online presence
- Correlate publicly available information
- Discover his contact details
- Determine his final location

Initial Investigation

The challenge begins with a screenshot.

Careful inspection of the image reveals a public profile URL:

https://www.komoot.com/user/5667624959835

This becomes our starting point.

Komoot Profile Analysis

Opening the Komoot profile reveals information associated with the account owner.

Further investigation eventually leads to another platform.

A linked GitHub profile is identified:

https://github.com/jiml33t

The username:

jiml33t

appears to be used consistently across platforms.

At this stage we have our primary OSINT target.

GitHub Enumeration

The next step is reviewing the GitHub profile for exposed information.

Public repositories and commits often contain:

- Usernames
- Emails
- Metadata
- Historical account details

One useful technique is reviewing commit patches directly.

GitHub allows commit data to be viewed in patch format.

Example:

https://github.com/jiml33t/jiml33t/commit/7b2c8e0a540c36f2e09da5945066020621d6a059.patch

Reviewing the patch reveals author metadata.

Email Discovery

Within the commit information, the email address becomes visible.

Recovered email:

jimleepro1@gmail.com

This provides a verified contact point associated with the target.

Phone Number Enumeration

The next challenge requires identifying the target's phone number.

Using the recovered email address, a message is sent.

An automated response is returned.

Contents:

Good day,

I will be absent from the office while I prepare for a marathon.

You can contact my on my phone for anything urgent.

Best Regards,

JL

0x4A4C

JIM LEE

CYBERSECURITY CONSULTANT

jimleepro1@gmail.com

L33T SECURITY

PENTESTING · RED TEAM · CONSULTING

The autoresponder provides additional confirmation that:

Jim Lee

is indeed the account owner.

The challenge answer can then be extracted from the contact details contained within the response.

Social Media Investigation

To continue the investigation, attention shifts to the target's social media activity.

A Threads profile is discovered:

https://www.threads.com/@jiml33t/post/DYCg8B1iMAl/media

The post contains an image.

At first glance, the image appears unrelated.

However, geolocation analysis often relies on:

- Architecture
- Street signs
- Public transport
- Road markings
- Language
- Urban infrastructure

Image Geolocation

Analyzing the image reveals several geographical indicators.

Using image analysis and geolocation techniques, the location is identified as:

Timișoara

Romania

This significantly narrows the search area.

Tram Station Identification

The challenge requires identifying the specific tram station visible in the image.

After comparing landmarks and transportation infrastructure within the city, the exact station is identified.

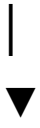
Recovered location:

Piața Gheorghe Domășneanu

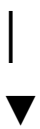
This represents the final location required to complete the investigation.

Investigation Path

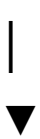
Conversation Screenshot



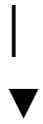
Komoot Profile



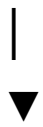
GitHub Profile



Commit Patch Analysis



Email Discovery



Autoresponder Investigation



Phone Number Recovery



Threads Profile

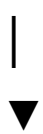
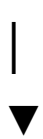


Image Analysis



Timișoara, Romania



Piața Gheorghe Domășneanu

Key OSINT Techniques Used

Profile Pivoting

Moving from one platform to another using:

- Shared usernames
- Linked accounts
- Public references

GitHub Metadata Analysis

Examining:

Commit History

Patch Files

Author Metadata

to recover exposed email addresses.

Email Intelligence

Using known email addresses to:

- Verify ownership
- Trigger autoresponders
- Gather additional contact information

Social Media Analysis

Reviewing public posts to identify:

- Travel locations
- Habits
- Interests
- Physical whereabouts

Geolocation

Using image-based OSINT techniques to identify:

- Cities
- Public infrastructure
- Transportation hubs
- Precise locations

Final Answers

Question	Answer
GitHub Username	jiml33t
Email Address	jimleepro1@gmail.com
Identified Person	Jim Lee
City	Timișoara
Country	Romania
Tram Station	Piața Gheorghe Domășneanu

Flag - TBCTF{Piața_Gheorghe_Domășneanu}